

BÁO CÁO THUYẾT MINH PHƯƠNG PHÁP XÂY DỰNG AI AGENT ĐA TÁC VỤ

Đội thi: Bảng C - Innovator | Tên đội: T1S_KMML

Tóm tắt giải pháp (Abstract)

Báo cáo này thuyết minh giải pháp xây dựng AI Agent đa tác vụ trên nền tảng mô hình ngôn ngữ lớn **Qwen3.5-4B-AWQ-4bit** để giải quyết bài toán trắc nghiệm đa ngành. Giải pháp vận hành hoàn toàn offline trong môi trường Docker Container theo đúng các tiêu chí kỹ thuật của cuộc thi. Nhóm tác giả đề xuất kiến trúc suy luận tối ưu kết hợp lập luận Chain-of-Thought (CoT), bộ trích xuất tiếng Việt phân biệt chữ hoa/thường nghiêm ngặt và cơ chế **Logit Margin Early Exit (Dừng sớm dựa trên độ phân vân)**. Hệ thống được điều khiển trực tiếp qua lớp thư viện vLLM LLMEngine cấp thấp với kỹ thuật gói đầu bất đồng bộ (Dynamic Pipelining). Kết quả thực nghiệm trên tập Public Test (463 câu hỏi) đạt độ chính xác cao **88.98%** với tốc độ xử lý vượt trội trung bình **1.42 giây/câu hỏi**.

1. Đặt vấn đề và Lựa chọn mô hình nền tảng

Yêu cầu cốt lõi của Bảng C - Innovator là phát triển một AI Agent có năng lực tự động xử lý và giải quyết chính xác các câu hỏi trắc nghiệm đa ngành phức tạp trong điều kiện **offline hoàn toàn (không có kết nối mạng)** và được đóng gói trọn vẹn trong môi trường Docker Container. Điều này đòi hỏi giải pháp phải giải quyết triệt để sự cân bằng giữa độ chính xác suy luận logic và tốc độ thực thi trên tài nguyên phần cứng giới hạn (GPU RTX 5060Ti / RTX 4060Ti / L4).

Nhóm tác giả đã lựa chọn mô hình nền tảng lượng tử hóa **Qwen3.5-4B-AWQ-4bit** (kích thước ~3.8GB). Việc lượng tử hóa giúp mô hình tiết kiệm tối đa dung lượng VRAM và bộ nhớ đệm KV Cache để chạy với batch size lớn, trong khi vẫn giữ vững năng lực suy luận ngữ cảnh vượt trội của dòng mô hình Qwen3.5. Để điều khiển mô hình ngoại tuyến hiệu quả nhất, hệ thống giao tiếp trực tiếp với lớp API LLMEngine của vLLM thay vì dựng API server cồng kềnh.

2. Kiến trúc luồng xử lý và Giải pháp thuật toán

Để đạt hiệu năng cao nhất, giải pháp tích hợp quy trình suy luận thích ứng thông minh bao gồm các bước xử lý chính:

2.1. Sơ đồ Luồng xử lý hệ thống (Pipeline Flowchart)

SƠ ĐỒ HỆ THỐNG XỬ LÝ VÀ LUỒNG Suy LUẬN AGENT - T1S_KMML



Sơ đồ kiến trúc xử lý của AI Agent

2.2. Cơ chế quyết định Logit Margin Early Exit

Thay vì luôn chạy biểu quyết nhiều luồng cho mọi câu hỏi gây lãng phí tài nguyên và kéo dài thời gian chạy, nhóm tác giả đã thiết kế cơ chế **Logit Margin Early Exit** để thích ứng theo độ khó của câu hỏi:

- 1. Luồng CoT chính (Luồng 0):** Hệ thống sinh lập luận Chain-of-Thought đầy đủ cho câu hỏi ở nhiệt độ thấp ($\text{temperature}=0.2$) để đảm bảo tính chính xác trong lập luận logic.
- 2. Confidence Check:** Thực hiện một request cực nhanh để sinh tiếp 1 token chốt đáp án bằng giải thuật Greedy ($\text{temperature}=0.0$) kèm tham số $\text{logprobs}=5$ để lấy phân phối xác suất của 5 token hàng đầu.
- 3. Tính toán Margin (Độ phân vân):** Hiệu xác suất giữa lựa chọn hàng đầu và lựa chọn thứ hai được tính bằng công thức:

$$\text{Margin} = P(\text{Lựa chọn 1}) - P(\text{Lựa chọn 2})$$

* **Early Exit (Margin ≥ 0.50):** Khi chênh lệch xác suất lớn, mô hình đã dứt khoát đưa ra đáp án và ít phân vân. Hệ thống lập tức thực hiện dừng sớm, lấy đáp án của Luồng 0 và kết thúc

câu hỏi (chỉ tiêu tốn 1 luồng sinh duy nhất). * **Biểu quyết đa số (Margin < 0.50)**: Khi mô hình bị phân vân (khoảng cách xác suất giữa hai lựa chọn hàng đầu dưới 50%), hệ thống kích hoạt song song Luồng 1 và Luồng 2 và tiến hành biểu quyết đa số (Majority Voting) giữa 3 luồng để đưa ra quyết định chuẩn xác nhất.

2.3. Bộ trích xuất tiếng Việt phân biệt chữ hoa - thường (Case-Sensitive Regex)

Do tiếng Việt tự nhiên chứa nhiều từ vựng viết thường trùng khớp hoàn toàn với các ký tự nhân lựa chọn trắc nghiệm (như 'ngày', 'việc', 'hành', 'về...'), việc sử dụng bộ lọc không phân biệt chữ hoa/thường (case-insensitive) sẽ dẫn đến lỗi bóc tách sai đáp án nghiêm trọng. Giải pháp đã tích hợp bộ lọc biểu thức chính quy **phân biệt nghiêm ngặt chữ hoa/chữ thường (case-sensitive)** kết hợp ranh giới từ dạng `\b[A-Z]\b` nhằm định vị chính xác và duy nhất chữ cái đáp án in hoa độc lập, loại bỏ hoàn toàn nhiễu từ ngữ tiếng Việt.

3. Tối ưu hóa hệ thống và Hiệu năng tính toán

Để vận hành mượt mà trong môi trường Docker Container biệt lập, hệ thống được tinh chỉnh hiệu năng tính toán: 1. **Gối đầu bất đồng bộ (Dynamic Pipelining)**: Tận dụng cơ chế xử lý hàng đợi của vLLM để đẩy các yêu cầu xử lý tiếp theo ngay khi bước trước vừa hoàn thành. vLLM sẽ thực hiện batching gối đầu các luồng tính toán đồng thời trên GPU, tối đa hóa công suất phần cứng. 2. **FlashAttention và Cấu hình KV Cache**: Sử dụng backend FLASH_ATTN kết hợp thiết lập `enable_prefix_caching=True` để chia sẻ tài nguyên bộ nhớ đệm KV Cache của prompt câu hỏi dùng chung giữa các luồng biểu quyết, giảm thiểu xử lý Prefill lặp lại. 3. **Thích ứng VRAM động**: Hệ thống tự động nhận diện dung lượng VRAM GPU và thiết lập mức sử dụng `gpu_memory_utilization` tối ưu (từ 0.85 đến 0.90) để đảm bảo không xảy ra lỗi tràn bộ nhớ (Out-Of-Memory).

4. Kết quả thực nghiệm

Giải pháp được đánh giá thực nghiệm trực tiếp trên toàn bộ tập dữ liệu kiểm thử công khai (Public Test) gồm 463 câu hỏi: * **Độ chính xác (Accuracy)**: Đạt tỷ lệ chính xác cao **88.98%** (chỉ sai 51 trên tổng số 463 câu hỏi trắc nghiệm đa lĩnh vực). * **Tốc độ thực thi (Throughput)**: Tổng thời gian hoàn thành suy luận là **657.46 giây**, đạt tốc độ trung bình **1.42 giây / câu hỏi**. * **Hiệu quả dừng sớm (Early Exit Rate)**: Đạt tỷ lệ dừng sớm **74.3%** trên tổng số câu hỏi, chứng minh thuật toán đã lược bỏ thành công việc sinh biểu quyết cho phần lớn các câu hỏi dễ và trung bình để đẩy tốc độ hệ thống lên tối đa.

5. Kết luận

Giải pháp AI Agent của nhóm tác giả đề xuất mang lại độ chính xác suy luận vượt trội cùng hiệu năng thực thi tối ưu. Việc kết hợp chặt chẽ giữa logic thuật toán Logit Margin thích ứng

và cấu hình vLLM offline đóng gói trong Docker Container tạo nên một giải pháp hoàn chỉnh, có độ ổn định cực cao và sẵn sàng đáp ứng vòng đánh giá Private Test tiếp theo.